Shama Perfume Store

API Documentation

For Android & iOS Mobile Teams

Field	Value
Domain	proly.ai
Supabase REST Base	https://[project].supabase.co/rest/v1
Edge Functions Base	https://[project].supabase.co/functions/v1
Storage Base	https://[project].supabase.co/storage/v1/object/public
External AI	https://openrouter.ai/api/v1 (model: openai/gpt-5.2)
Delivery API	https://app.vanex.ly/api/v1
Date	April 09, 2026
Version	1.0.0

This document lists every API endpoint, database table, request/response schema, and environment variable required to build native mobile apps against the Shama backend.

Table of Contents

1. Authentication

1.1 Supabase Anonymous Key (Public Reads)

All public reads (products, approved reviews, tracking) use the Supabase **anon key** in the Authorization header. Row-Level Security (RLS) controls what the anon key can access.

```
Header: apikey: {SUPABASE_ANON_KEY}
Header: Authorization: Bearer {SUPABASE_ANON_KEY}
```

1.2 Admin Authentication (Username + Password)

Admin login checks the **users** table directly (plain-text password). No JWT is returned — the admin session is cached in localStorage for 24 hours. For mobile apps, store the admin session token in secure storage.

POST	/rest/v1/users?select=*	Admin login query

Request body (filter params as query string):

```
{
"username": "admin",
"password": "admin123"
}

// Supabase query equivalent:
// SELECT * FROM users WHERE username=? AND password=? AND is_active=true
```

Response:

```
{
"id": "uuid",
"username": "admin",
"role": "admin",
"is_active": true,
"created_at": "2024-02-22T18:11:00Z"
}
```

1.3 Supabase Auth (End Users)

User-facing features (reviews, orders by email, tracking) use Supabase Auth. Email/password login, OTP, and Google OAuth are supported. Use the Supabase SDK for your platform.

Method	Endpoint	Description
POST	/auth/v1/signup	Register with email + password
POST	/auth/v1/token?grant_type=password	Login — returns access_token + refresh_token
POST	/auth/v1/otp	Send OTP to email
POST	/auth/v1/token?grant_type=refresh_token	Refresh session
POST	/auth/v1/logout	Logout — invalidate token
GET	/auth/v1/user	Get current authenticated user

2. Frontend Routes (Page Navigation)

These are the React Router paths. The mobile app should implement equivalent screens.

Path	Screen	Auth Required	Description
/	Home	No	Landing page with featured products and hero banner
/collection	Collection	No	Browse/search/filter all perfumes
/product/:id	Product Detail	No	Full product page: images, notes, reviews, variants
/gift-sets	Gift Sets	No	Pre-built gift set products
/samples	Samples	No	Sample-size products (3ml–30ml)
/quiz	Fragrance Quiz	No	Interactive quiz → AI perfume recommendations
/ai-finder	AI Finder	No	Natural language AI perfume search
/wishlist	Wishlist	No	Locally stored wishlist (localStorage)
/my-orders	My Orders	Email only	Order history lookup by email address
/track-order	Order Tracking	No	Track any order by ID or email
/settings	Email Prefs	Supabase Auth	Manage email notification preferences
/login	Admin Login	No	Admin-only login page
/admin	Admin Dashboard	Admin pass	Full admin: orders, products, reviews, promos

3. Database Schema — All Tables

All tables live in the Supabase Postgres instance. Access via REST: `https://[project].supabase.co/rest/v1/{table_name}`. RLS is enabled on every table.

3.1 perfumes

Column	Type	Notes
id	UUID PK	Auto-generated
name	text	English name
name_ar	text	Arabic name
price	decimal	Base price (LYD)
description	text	English description
description_ar	text	Arabic description
fragrance_notes	JSON	{top:[...], middle:[...], base:[...]}
fragranceNotes_ar	JSON	Arabic fragrance notes
size	text	e.g. '100ml'
type	text	bottle | sample | gift
rating	decimal	Auto-updated by trigger on review changes
gender	text	men | women | unisex
stock_quantity	int	Current stock level
is_active	bool	Public visibility flag
has_samples	bool	True if sample variants exist
has_bottle_sizes	bool	True if bottle size variants exist
created_at	timestamptz	
updated_at	timestamptz	

3.2 perfume_images

Column	Type	Notes
id	UUID PK	
perfume_id	UUID FK	References perfumes.id
image_url	text	Full public URL to image
image_name	text	Original filename
file_path	text	Supabase Storage path
file_size	int	Bytes
mime_type	text	e.g. image/jpeg
is_primary	bool	True = main product image
display_order	int	0-based sort order

Column	Type	Notes
created_at	timestamptz	
updated_at	timestamptz	

3.3 perfume_samples

Column	Type	Notes
id	UUID PK	
perfume_id	UUID FK	References perfumes.id
size	text	3ml | 5ml | 10ml | 15ml | 20ml | 25ml | 30ml
price	decimal	Price for this sample size
stock_quantity	int	
is_active	bool	
created_at	timestamptz	

3.4 perfume_bottle_sizes

Column	Type	Notes
id	UUID PK	
perfume_id	UUID FK	References perfumes.id
size	text	30ml | 50ml | 75ml | 100ml | 125ml | 150ml | 200ml
price	decimal	
stock_quantity	int	
is_active	bool	
created_at	timestamptz	

3.5 orders

Column	Type	Notes
id	UUID PK	
first_name	text	
last_name	text	
email	text	Customer email
phone	text	e.g. +218913456789
city	text	City name
place_name	text	Sub-city / neighbourhood
total	decimal	Final total after discount
subtotal	decimal	Before discount
discount_amount	decimal	Amount saved via promo

Column	Type	Notes
delivery_fee	decimal	Shipping cost
order_date	timestamptz	When order was placed
items	JSON[]	Array of OrderItem objects
status	text	pending | confirmed | processing | shipped | delivered | accepted | returned
payment_method	text	cod | bank_transfer
transfer_proof_url	text	Image URL of bank transfer receipt
vanex_city_id	int	Vanex city ID
vanex_sub_city_id	int	Vanex sub-city ID
vanex_package_code	text	Tracking code from Vanex
promo_code_id	UUID FK	Applied promo code
promo_code_snapshot	JSON	Snapshot of promo at time of order
processed_at	timestamptz	When status last changed
created_at	timestamptz	

3.6 reviews

Column	Type	Notes
id	UUID PK	
perfume_id	UUID FK	References perfumes.id
user_id	text	Supabase Auth user ID
user_email	text	
rating	int	1–5
comment	text	Review text
status	text	pending (default) | approved
created_at	timestamptz	
updated_at	timestamptz	

3.7 promo_codes

Column	Type	Notes
id	UUID PK	
code	text UNIQUE	e.g. SHAMA10
discount_type	text	fixed | percentage
discount_value	decimal	Amount or percent
max_discount	decimal	Cap for percentage discounts
min_order_total	decimal	Minimum cart value required

Column	Type	Notes
scope	text	all_products | specific_products
scope_product_ids	JSON	Array of product UUIDs (if specific)
is_active	bool	
expires_at	timestamptz	Nullable
usage_limit	int	Total uses allowed
usage_limit_per_user	int	Per-user cap
usage_count	int	Current total uses
created_at	timestamptz	

3.8 memories

Column	Type	Notes
id	UUID PK	
perfume_id	UUID	Associated perfume
perfume_name	text	Denormalized for display
memory_text	text	User's scent memory story
author_name	text	
status	text	pending | approved
created_at	timestamptz	

3.9 user_events

Column	Type	Notes
id	UUID PK	
user_id	text	Supabase Auth user ID
event_type	text	See event type list below
event_data	JSON	Payload — varies by event type
created_at	timestamptz	

Event types:

Event Type	Trigger
product_view	User opens product detail page
cart_add / cart_remove	Item added/removed from cart
wishlist_add / wishlist_remove	Item added/removed from wishlist
search_query	User types in search bar
ai_search_query	User submits AI/smart search
chatbot_query	User sends message to AI chatbot
quiz_completion	User completes fragrance quiz

Event Type	Trigger
scent_dna_generated	User generates Scent DNA card
purchase	Successful order placed

3.10 user_taste_profiles

Column	Type	Notes
id	UUID PK	
user_id	text UNIQUE	Supabase Auth user ID
scent_families	JSON	[{name, score}] — e.g. oriental 0.85
preferred_notes	JSON	[{note, score}] — e.g. oud 0.9
price_range	JSON	{min, max, confidence}
gender_pref	JSON	{value, confidence}
occasion_pref	JSON	[{occasion, score}]
intensity_pref	JSON	{value, confidence}
total_events_analyzed	int	
last_event_at	timestamptz	
profile_version	int	Increments on each re-analysis
created_at	timestamptz	
updated_at	timestamptz	

3.11 Email Tables (email_preferences, email_queue, email_log)

Table	Key Columns	Purpose
email_preferences	user_id, email_enabled, language_pref, unsubscribe_token, weekly_digest, monthly_digest, new_product_alerts, re_eng	User email opt-in settings
email_queue	user_id, email_type, to_email, subject_en, subject_ar, body_html, product_ids, promo_code_id, status	Pending emails to be sent
email_log	user_id, email_type, subject, product_ids, resend_message_id, sent_at	Sent email history
product_announcements	product_id, user_id, created_at	Prevents duplicate new-product emails
processing_metadata	key (unique), value (JSON), updated_at	Nightly job state timestamps

4. Products API

GET	/rest/v1/perfumes	List all active products (paginated)

Query parameters:

Param	Type	Default	Description
select	string	*	Fields to return; use * for all
is_active	eq.true	required	Only return visible products
order	string	created_at.desc	Sort order
limit	int	20	Page size
offset	int	0	Pagination offset
type	text	—	Filter: eq.bottle | eq.sample | eq.gift
gender	text	—	Filter: eq.men | eq.women | eq.unisex
price	range	—	gte.100&price=lte.300

Example URL:

```
GET /rest/v1/perfumes?select=*&is_active=eq.true&type=eq.bottle&order=created_at.desc&limit=20
&offset=0
```

Response (array of perfume objects):

```
[
{
"id": "a1b2c3d4-...",
"name": "Velvet Rose",
"name_ar": "[redacted]",
"price": 200.00,
"description": "A rich floral with warm amber base...",
"fragrance_notes": {
"top": ["Rose", "Peony"],
"middle": ["Magnolia", "Jasmine"],
"base": ["Cedarwood", "Amber", "Musk"]
},
"size": "100ml",
"type": "bottle",
"rating": 4.9,
"gender": "women",
"stock_quantity": 8,
"is_active": true,
"has_samples": true,
"has_bottle_sizes": true
}
]
```

GET	/rest/v1/perfumes?id=eq.{id}	Get single product by ID

```
GET /rest/v1/perfumes?id=eq.a1b2c3d4-...&select=*&is_active=eq.true
Header: Accept: application/vnd.pgrst.object+json // returns object not array
```

GET	/rest/v1/perfume_images?perfume_id=eq.{id}	Get images for a product

```
GET /rest/v1/perfume_images?perfume_id=eq.{id}&order=display_order.asc&select=*
```

GET	/rest/v1/perfume_samples?perfume_id=eq.{id}	Get sample sizes for a product

```
GET /rest/v1/perfume_samples?perfume_id=eq.{id}&is_active=eq.true&order=size.asc
```

GET	/rest/v1/perfume_bottle_sizes?perfume_id=eq.{id}	Get bottle size variants

```
GET /rest/v1/perfume_bottle_sizes?perfume_id=eq.{id}&is_active=eq.true&order=size.asc
```

Search Products:

GET	/rest/v1/perfumes	Text search on name / name_ar

```
GET /rest/v1/perfumes?or=(name.ilike.*rose*,name_ar.ilike.*■■■*)&is_active=eq.true
```

5. Orders API

POST	/rest/v1/orders	Place a new order

Request body:

```
{
"first_name": "John",
"last_name": "Doe",
"email": "john@example.com",
"phone": "+218913456789",
"city": "Tripoli",
"place_name": "Hay Andalos",
"vanex_city_id": 1,
"vanex_sub_city_id": 5,
"total": 350.00,
"subtotal": 400.00,
"discount_amount": 50.00,
"delivery_fee": 0,
"payment_method": "cod",
"promo_code_id": "uuid-or-null",
"promo_code_snapshot": { "code": "SHAMA10", "discount": 50 },
"items": [
{
"id": "product-uuid",
"name": "Velvet Rose",
"price": 200,
"size": "100ml",
"quantity": 2,
"image": "https://..."
}
],
"order_date": "2026-04-09T12:34:56Z",
"status": "pending"
}
```

GET	/rest/v1/orders?email=eq.{email}	Get orders by customer email

```
GET /rest/v1/orders?email=eq.john%40example.com&order=created_at.desc
```

GET	/rest/v1/orders?id=eq.{id}	Get single order by ID

```
GET /rest/v1/orders?id=eq.{order-uuid}
```

GET	/rest/v1/orders	Track order (by ID or email)

```
# By order ID:
GET /rest/v1/orders?id=eq.{uuid}&select=id,status,items,order_date,total,vanex_package_code

# By email + date range:
GET /rest/v1/orders?email=eq.{email}&order=order_date.desc
```

Order status values:

Status	Meaning
pending	Just placed, awaiting confirmation
confirmed	Admin confirmed the order
processing	Order is being prepared
shipped	Handed to Vanex — package code assigned
delivered	Delivered to customer
accepted	Customer accepted delivery
returned	Order returned

6. Reviews API

GET	/rest/v1/reviews	Get approved reviews for a product

```
GET /rest/v1/reviews?perfume_id=eq.{id}&status=eq.approved&order=created_at.desc
```

GET	/rest/v1/reviews	Get a user's own review for a product

```
GET /rest/v1/reviews?perfume_id=eq.{id}&user_id=eq.{userId}
Header: Authorization: Bearer {user_access_token}
```

POST	/rest/v1/reviews	Submit a new review

Request body:

```
{
"perfume_id": "product-uuid",
"user_id": "auth-user-id",
"user_email": "user@example.com",
"rating": 5,
"comment": "Amazing fragrance, long-lasting!"
}
// status defaults to pending; AI evaluates it server-side
```

Note: After submission the AI evaluator (evaluateReview) may auto-approve the review if content passes moderation. Otherwise it stays pending for admin review.

PATCH	/rest/v1/reviews?id=eq.{id}	Approve a review (admin only)

```
PATCH /rest/v1/reviews?id=eq.{reviewId}
Header: apikey: {SUPABASE_SERVICE_ROLE_KEY}
Body: { "status": "approved" }
```

DELETE	/rest/v1/reviews?id=eq.{id}	Delete a review (admin only)

```
DELETE /rest/v1/reviews?id=eq.{reviewId}
Header: apikey: {SUPABASE_SERVICE_ROLE_KEY}
```

7. Promo Codes API

GET	/rest/v1/promo_codes?code=eq.{code}	Validate a promo code

```
GET /rest/v1/promo_codes?code=eq.SHAMA10&is_active=eq.true
```

Client-side validation logic (implement in mobile app):

```
1. Check expires_at — reject if in the past
2. Check usage_count < usage_limit
3. Check cart total >= min_order_total
4. If scope=specific_products: ensure at least one cart item is in scope_product_ids
5. Calculate discount:
- type=percentage: discount = eligibleSubtotal * (discount_value / 100)
capped at max_discount (if set)
- type=fixed: discount = discount_value (cannot exceed eligibleSubtotal)
6. After successful order: PATCH usage_count = usage_count + 1
```

PromoCode object:

```
{
"id": "uuid",
"code": "SHAMA10",
"discount_type": "percentage",
"discount_value": 10,
"max_discount": 100,
"min_order_total": 50,
"scope": "all_products",
"scope_product_ids": [],
"is_active": true,
"expires_at": "2026-12-31T23:59:59Z",
"usage_limit": 1000,
"usage_limit_per_user": 5,
"usage_count": 42
}
```

8. AI / Smart Search API

All AI features call OpenRouter directly from the client using the VITE_OPENROUTER_API_KEY. The mobile app should use the same key securely (store in a secure vault, not bundled). Model: `openai/gpt-5.2`

8.1 Chatbot (Streaming)

POST	https://openrouter.ai/api/v1/chat/completions	AI Chatbot

```
{
"model": "openai/gpt-5.2",
"stream": true,
"messages": [
{
"role": "system",
"content": "[Full product catalog injected — ~5 min cache]"
},
{
"role": "user",
"content": "What perfume is best for evenings?"
}
]
}
```

Response: Server-Sent Events (SSE) stream. Each chunk is a delta. Accumulate delta.content to reconstruct the full response.

8.2 Smart Search / AI Finder

POST	https://openrouter.ai/api/v1/chat/completions	Smart Search

```
// Prompt asks the AI to return structured JSON of matching products:
{
"model": "openai/gpt-5.2",
"messages": [
{ "role": "system", "content": "[catalog context]" },
{ "role": "user", "content": "Find me a woody oud for evening" }
]
}

// Response shape (parsed from AI text):
[
{
"name": "Velvet Rose",
"matchScore": 92,
"reason": "Rich oud and cedarwood base ideal for evenings"
}
]
```

8.3 Fragrance Quiz Recommendations

```
// Quiz answers format:
```

```
{
"gender": "women",
"scentFamily": "floral",
"occasion": "date_night",
"intensity": "moderate",
"season": "summer",
"budget": "100-250"
}

// AI returns top-3 recommendations:
[
{
"name": "Velvet Rose",
"matchScore": 95,
"reason": "Light floral with soft musk — perfect for summer dates"
}
]
```

8.4 Scent DNA Card Generation

```
// Input: quiz answers
// Output:
{
"archetype": "The Desert Wanderer",
"archetypeAr": "■■■■■■■■ ■■ ■■■■■■■■",
"families": [
{ "name": "Oriental", "nameAr": "■■■■■", "percent": 65 },
{ "name": "Woody", "nameAr": "■■■■■", "percent": 35 }
],
"signatureNotes": ["Oud", "Amber", "Sandalwood"],
"bestTime": "Evening",
"bestTimeAr": "■■■■■■■",
"bestSeason": "Winter",
"bestSeasonAr": "■■■■■■■"
}
```

9. Vanex Logistics API

Vanex is the Libyan delivery partner. Base URL: `https://app.vanex.ly/api/v1`. Requires Bearer token: `VITE_VANEX_TOKEN`.

GET	`https://app.vanex.ly/api/v1/city/all`	Get all delivery cities

Headers: `Authorization: Bearer {VANEX_TOKEN}`

Response:

```
[
{
"id": 1,
"name": "Tripoli",
"price": 0,
"branch": 1,
"code": "TP",
"locations": [
{ "id": 5, "name": "Hay Andalos", "parent_city": 1, "price": 50 },
{ "id": 6, "name": "Ain Zara", "parent_city": 1, "price": 50 }
]
}
]
```

POST	`https://app.vanex.ly/api/v1/customer/package`	Create shipping package

Request body:

```
{
"type": 1,
"reciever": "John Doe",
"phone": "+218913456789",
"city": 1,
"address_child": 5,
"address": "Hay Andalos",
"price": 350,
"payment_methode": "cash",
"paid_by": "customer",
"description": "■■■■ ■■■",
"qty": 1,
"height": 20,
"leangh": 20,
"width": 20
}
```

Response: `{ "package_code": "TP-12345" }`

GET	`https://app.vanex.ly/api/v1/tracking?code={code}`	Track a package (public, no auth)

```
GET https://app.vanex.ly/api/v1/tracking?code=TP-12345
// No Authorization header needed
```

10. Images & Storage

10.1 Public Image URLs

All product images are public. URL format:

```
https://[project].supabase.co/storage/v1/object/public/perfume-images/{file_path}
```

10.2 Upload Image (Admin)

POST	`/storage/v1/object/perfume-images/{path}`	Upload perfume image

```
POST /storage/v1/object/perfume-images/{perfume_id}/{timestamp}.jpg
Header: apikey: {SERVICE_ROLE_KEY}
Header: Authorization: Bearer {SERVICE_ROLE_KEY}
Header: Content-Type: image/jpeg
Body: [binary image data]

// Then INSERT metadata into perfume_images table
```

10.3 Gift Preview Storage Bucket

AI-generated gift box preview images are stored in the `gift-previews` bucket. Same URL format applies.

11. Memories (Scent Wall)

GET	/rest/v1/memories	Get approved scent memories

```
GET /rest/v1/memories?status=eq.approved&order=created_at.desc&limit=40
```

POST	/rest/v1/memories	Submit a scent memory

```
{
"perfume_id": "uuid",
"perfume_name": "Velvet Rose",
"memory_text": "Reminds me of my grandmother's garden...",
"author_name": "Sarah",
"status": "pending"
}
```

12. User Events & Tracking

Fire-and-forget event tracking. Used by the AI personalization engine to build taste profiles. Call this for every meaningful user action.

POST	/rest/v1/user_events	Track a user event

Examples for each event type:

```
// product_view
{ "user_id": "auth-uid", "event_type": "product_view",
"event_data": { "product_id": "uuid", "product_name": "Velvet Rose", "price": 200 } }

// cart_add
{ "user_id": "auth-uid", "event_type": "cart_add",
"event_data": { "product_id": "uuid", "quantity": 1, "price": 200 } }

// search_query
{ "user_id": "auth-uid", "event_type": "search_query",
"event_data": { "query": "oud evening" } }

// quiz_completion
{ "user_id": "auth-uid", "event_type": "quiz_completion",
"event_data": { "answers": { "gender": "women", "occasion": "date_night" } } }

// purchase
{ "user_id": "auth-uid", "event_type": "purchase",
"event_data": { "order_id": "uuid", "total": 350, "items_count": 2 } }
```

13. Custom Gift Orders

POST	/rest/v1/custom_gift_orders	Place a custom gift order

```
{
"user_id": "auth-uid-or-null",
"products": [
{ "id": "uuid", "name": "Velvet Rose", "price": 200 }
],
"occasion": "birthday",
"box_color": "gold",
"wrapping_style": "ribbon",
"message_card": "Happy Birthday!",
"recipient_name": "Sarah",
"delivery_date": "2026-04-15",
"generated_image_url": "https://...",
"image_style": "realistic",
"total_price": 250,
"status": "pending"
}
```

14. Email Preferences

GET	/rest/v1/email_preferences?user_id=eq.{uid}	Get user's email settings

```
GET /rest/v1/email_preferences?user_id=eq.{auth-uid}
Header: Authorization: Bearer {user_access_token}
```

PATCH	/rest/v1/email_preferences?user_id=eq.{uid}	Update email preferences

```
PATCH /rest/v1/email_preferences?user_id=eq.{auth-uid}
Header: Authorization: Bearer {user_access_token}
Body:
{
"email_enabled": true,
"language_pref": "ar",
"weekly_digest": true,
"new_product_alerts": false
}
```

15. Supabase Edge Functions

Edge functions run on Deno at Supabase. They use the service-role key internally. They are typically called server-to-server or via cron; mobile apps should NOT call most of these directly except where noted.

15.1 send-email

POST	/functions/v1/send-email	Processes email_queue, sends via Resend API.

Usage: Server/cron only

Request:

```
{}
```

Response:

```
{"sent":5,"failed":0,"skipped":1}
```

15.2 analyze-user-profile

POST	/functions/v1/analyze-user-profile	Analyzes user_events and upserts taste profiles.

Usage: Server/cron only

Request:

```
{"user_ids":["uuid1","uuid2"]}
```

Response:

```
{"results":[{"user_id":"uuid","success":true}]}
```

15.3 match-new-products

POST	/functions/v1/match-new-products	Match new products to user profiles, queue announcement emails.

Usage: Called by admin after adding products

Request:

```
{"product_ids":["uuid"]}
```

Response:

```
{"matched":42}
```

15.4 nightly-orchestrator

POST	/functions/v1/nightly-orchestrator	Runs all background jobs: profiles, digests, re-engagement.

Usage: Cron: runs nightly

Request:

```
{}
```

Response:

```
{"success":true,"log":["..."]}
```

15.5 unsubscribe

GET	`/functions/v1/unsubscribe`	Unsubscribe user from emails via token in email link.

Usage: Public — opened from email link

Request:

```
?token={unsubscribe_token}
```

Response:

```
HTML confirmation page
```

16. Telegram Bot Services

The Telegram bot is a separate Node.js process that connects to the same Supabase project using the service-role key. It uses Telegraf and Gemini AI. Mobile apps do not interact with the Telegram bot directly.

Function	Operation	Description
searchProducts(query, limit)	SELECT from perfumes	Full-text product search
getProductById(id)	SELECT single	Get product details
getProductByName(name)	SELECT ilike	Find by name
createProduct(draft)	INSERT into perfumes	Admin: add new product
updateProduct(id, changes)	UPDATE perfumes	Admin: edit product fields
deleteProduct(id)	DELETE perfumes	Admin: remove product
fetchOrders(options)	SELECT orders	List orders with filters
getOrderById(id)	SELECT single order	Look up order by ID or short ID
getAnalytics()	Aggregate queries	Revenue, orders, top products
analyzeImage(buffer)	Gemini Vision API	AI analysis of product photo
transcribeVoice(buffer)	Gemini Audio API	Voice message transcription

17. Environment Variables

17.1 Mobile App (Client)

Variable	Where to Get	Required For
SUPABASE_URL	Supabase project Settings > API	All DB/auth/storage calls
SUPABASE_ANON_KEY	Supabase project Settings > API	All public reads
OPENROUTER_API_KEY	openrouter.ai/keys	AI chatbot, smart search, quiz
VANEX_TOKEN	Vanex merchant dashboard	Delivery cities + create packages

17.2 Edge Functions (Server)

Variable	Required For
SUPABASE_URL	DB access
SUPABASE_SERVICE_ROLE_KEY	Bypass RLS for admin operations
OPENROUTER_API_KEY	AI personalization (analyze-user-profile, match-new-products)
RESEND_API_KEY	Sending emails via Resend

17.3 Telegram Bot

Variable	Required For
SUPABASE_URL	DB access
SUPABASE_SERVICE_KEY	Bypass RLS
GEMINI_API_KEY	AI responses, image analysis
GROQ_API_KEY	Fast inference fallback
TELEGRAM_BOT_TOKEN	Telegram API
ADMIN_CHAT_IDS	Restrict admin commands to ID 586087143

18. Complete Data Type Reference

18.1 OrderItem

```
{
"id": "string", // product UUID or variant ID
"name": "string", // English product name
"price": "number", // unit price in LYD
"size": "string", // e.g. "100ml", "5ml"
"quantity": "number", // number of units
"image": "string" // primary image URL
}
```

18.2 Product (Full)

```
{
"id": "uuid",
"name": "string",
"name_ar": "string",
"price": "number",
"description": "string",
"description_ar": "string",
"fragrance_notes": {
"top": ["string"],
"middle": ["string"],
"base": ["string"]
},
"fragranceNotes_ar": { "top": [], "middle": [], "base": [] },
"size": "string",
"type": "bottle | sample | gift",
"rating": "number",
"gender": "men | women | unisex",
"stock_quantity": "number",
"is_active": "boolean",
"has_samples": "boolean",
"has_bottle_sizes": "boolean",
"images": "PerfumeImage[]",
"samples": "PerfumeSample[]",
"bottle_sizes": "PerfumeBottleSize[]"
}
```

18.3 SmartSearchResult

```
{
"product": "Product", // full product object
"reason": "string", // 1-2 sentence match explanation
"matchScore": "number" // 60–99
}
```

18.4 ScentDNACard

```
{
"archetype": "string",
```

```
"archetypeAr": "string",
"families": [
{ "name": "string", "nameAr": "string", "percent": "number" }
],
"signatureNotes": ["string"],
"bestTime": "string",
"bestTimeAr": "string",
"bestSeason": "string",
"bestSeasonAr": "string"
}
```

18.5 TasteProfile

```
{
"scent_families": [{ "name": "string", "score": "number" }],
"preferred_notes": [{ "note": "string", "score": "number" }],
"price_range": { "min": "number", "max": "number", "confidence": "number" },
"gender_pref": { "value": "string", "confidence": "number" },
"occasion_pref": [{ "occasion": "string", "score": "number" }],
"intensity_pref": { "value": "string", "confidence": "number" },
"total_events_analyzed": "number",
"profile_version": "number"
}
```

For questions about implementation, contact the Shama backend team. All endpoints are HTTPS-only. Never embed the service-role key in client apps — use the anon key for all client-side requests.